

AdaBoost

01 April 2026 14:57

AdaBoost (Adaptive Boosting):

An ensemble method that builds multiple weak learners (usually decision stumps) sequentially, where each new model focuses more on the mistakes of the previous ones.

Instead of training all models independently:

- Model 1 makes mistakes
- Model 2 focuses more on those mistakes
- Model 3 focuses on remaining errors

Final model = combination of all

Analogy

- A teacher gives a test
- Students solve it

Round 1:

- Some questions are wrong

Round 2:

- Teacher says: "Focus more on the questions you got wrong"

Round 3:

- Again focus on remaining weak areas
- Final score improves step by step

AdaBoost Working

Step 1: Initialize weights

- All data points get equal weight
- If 100 points $\rightarrow$ each weight = $1/100$

Step 2: Train first weak learner

- Usually a decision stump (1-level tree)
- Predict on all data (To identify: Which points are correctly classified and Which are misclassified, then we use this to Compute error and Update weights)

Step 3: Compute error

Error = sum of weights of misclassified points

Step 4: Compute model weight (IMPORTANT)

- If model performs well $\rightarrow$ higher importance

- If poor $\rightarrow$ lower importance

Mathematical intuition (core formula)

Model weight:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{error}}{\text{error}} \right)$$

If error is small:

- α is large $\rightarrow$ model is important

If error is large:

- α is small $\rightarrow$ model less important

Step 5: Update data weights

- Misclassified points $\rightarrow$ **increase weight**
- Correct points $\rightarrow$ **decrease weight**

So next model focuses on “hard” points

$$w_i \leftarrow w_i \cdot e^{\pm \alpha}$$

- Misclassified $\rightarrow$ multiply by $e^{+\alpha}$ (increase)
- Correct $\rightarrow$ multiply by $e^{-\alpha}$ (decrease)

Step 6: Repeat

- Train next stump using updated weights
- Continue for many rounds

Step 7: Final prediction

Weighted combination of all models:

- Classification $\rightarrow$ weighted vote

Key parameters

1. n_estimators

- Number of weak learners
- More $\rightarrow$ better (up to a point)

2. learning_rate

- Controls contribution of each model

Smaller learning_rate (e.g., 0.1):

- Each weak learner contributes less to the final model.
- The algorithm adjusts weights slowly.

- **Pros:** More stable, less chance of overfitting.
- **Cons:** Might need **more weak learners** ($n_estimators$) to achieve good accuracy.

Larger learning_rate (e.g., 1.0):

- Each weak learner contributes more strongly.
- The algorithm adapts faster.
- **Pros:** Fewer models may be needed; faster convergence.
- **Cons:** Can **overfit**, especially if data is noisy.

if we set `learning_rate` low, increase `n_estimators` to compensate.
 If you want faster results but risk overfitting, increase `learning_rate` but keep `n_estimators` moderate.

3. Base estimator

- Usually *decision stump*

Example

Suppose we have 5 data points:

Point	Feature (x)	Label (y)
A	1	0
B	2	1
C	3	0
D	4	1
E	5	0

Step 1: Initialize weights

All weights equal:

$$A = 0.2$$

$$B = 0.2$$

$$C = 0.2$$

$$D = 0.2$$

$$E = 0.2$$

Step 2: Train first weak learner (stump)

Suppose model predicts:

Point	True (y)	Predicted	Correct?
-------	----------	-----------	----------

A	0	0	✓
B	1	0	✗
C	0	0	✓
D	1	1	✓
E	0	1	✗

Step 3: Compute Weighted error

Error = sum of weights of wrong points

$$\text{error} = \sum w_i \cdot I(y_i \neq \hat{y}_i)$$

- $w_i \rightarrow$ current weight of data point i
- $y_i \rightarrow$ true label of point i
- $\hat{y}_i \rightarrow$ prediction of the current weak learner
- $I(y_i \neq \hat{y}_i) \rightarrow$ indicator function:
 - 1 if the point is misclassified, 0 if correct

$$\text{error} = 0.2(B) + 0.2(E) = 0.4$$

Step 4: Compute model weight (α)

$$\begin{aligned} \alpha &= 0.5 \cdot \ln\left(\frac{1 - 0.4}{0.4}\right) \\ &= 0.5 \cdot \ln(0.6/0.4) \approx 0.2027 \end{aligned}$$

Step 5: Update weights

$$w_i \leftarrow w_i \cdot \exp(\alpha \cdot I(\text{misclassified}))$$

Before normalization:

Point	Old (w_i)	Correct?	New (before normalization)
A	0.2	✓	$0.2 \times 1 = 0.2$
B	0.2	✗	$0.2 \times \exp(0.2027) \approx 0.244$
C	0.2	✓	0.2
D	0.2	✓	0.2
E	0.2	✗	$0.2 \times \exp(0.2027) \approx 0.244$

Normalize (sum = 1)

$$w_i \leftarrow \frac{w_i}{\sum w_i}$$

$$w_i \leftarrow \frac{w_i}{\sum_j w_j}$$

Normalize Weights

- Total = $0.2 + 0.244 + 0.2 + 0.2 + 0.244 = 1.088$
- Divide each by 1.088:

Point Normalized (w_i)

A $0.2 / 1.088 \approx 0.184$

B $0.244 / 1.088 \approx 0.224$

C $0.2 / 1.088 \approx 0.184$

D $0.2 / 1.088 \approx 0.184$

E $0.244 / 1.088 \approx 0.224$

- Misclassified points (B, E) → **weights increased** → 0.224
- Correctly classified points (A, C, D) → **weights decreased** → 0.184

Next model will:

- Focus heavily on B and E

Step 6: Train second learner

Because B and E have high weight:

- Model tries harder to classify B and E correctly

So, the Learning shifts toward **hard cases**

